
RENTEASE — HOTEL LISTING & MANAGEMENT SYSTEM

DATABASE MANAGEMENT SYSTEM

Project Report

On

Course: Database Management Systems (UCS310)

Submitted To:

Dr. Abhishelly Sharma

Submitted By:

Vaibhav Singhal – 1024030972

Abhinav Parashar – 1024031165

Ajay Bhatti – 1024030989



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Table of Contents

1. Introduction
2. Entity-Relationship Diagram
3. ER Analysis
4. Conversion of ER to Tables
5. Normalization
6. Functional Dependencies after Normalization
7. SQL Queries (DDL & DML)
8. Table Structure
9. Project Screenshots
10. PL/SQL Components
11. Conclusion

1. Introduction

RentEase is a full-stack web application designed to simplify hotel and accommodation discovery and listing management. The platform enables property owners to list their accommodations and allows users to browse, review, and interact with listings. Built on a Node.js and Express backend with MongoDB as the primary data store, the system manages users, listings, and reviews through well-defined data relationships.

This report documents the complete relational database design for RentEase, demonstrating how the system's data architecture can be formally modeled using Entity-Relationship diagrams, converted to normalized relational tables, and implemented using SQL DDL and DML operations.

Key Features:

- User authentication and account management (register, login, logout)
- Property listing management (create, read, update, delete listings)
- Review system with ratings for each listing
- Image upload support via Cloudinary integration
- Interactive map display using Mapbox for listing locations
- Category-based filtering (Resort, Apartment, Houseboat, Spa, Apart Hotel)

Problem Statement:

Travelers frequently struggle to find reliable accommodation options with transparent reviews and pricing. Property owners lack a centralized platform to manage their listings and receive guest feedback. RentEase addresses both needs by providing a unified system for listing management, user authentication, and review tracking.

Objectives:

- Design a complete ER model for the hotel listing and review system
- Convert the ER diagram to a normalized relational schema
- Normalize the database up to Third Normal Form (3NF)
- Implement SQL DDL and DML operations for all entities
- Write analytical queries using JOIN, GROUP BY, HAVING, and subqueries
- Demonstrate referential integrity through foreign key constraints

Technology Stack:

- Backend: Node.js with Express.js framework
- Database: MongoDB (application layer) / PostgreSQL with raw SQL (relational model)
- Frontend: EJS templating engine with Bootstrap 5
- Cloud Storage: Cloudinary for image management
- Mapping: Mapbox GL JS for geolocation features
- Authentication: Passport.js with local strategy

2. Entity-Relationship Diagram

The ER diagram for RentEase models the real-world entities involved in a hotel listing and review platform. The central entity is LISTING, which connects USER (as owner) and REVIEW together.

Entities and Their Attributes:

1. USER

Attribute	Data Type	Constraint
user_id	INT / ObjectId	Primary Key
username	VARCHAR(255)	NOT NULL, UNIQUE
email	VARCHAR(255)	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL

2. LISTING

Attribute	Data Type	Constraint
listing_id	INT / ObjectId	Primary Key
title	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
image_url	TEXT	DEFAULT (Unsplash URL)
image_filename	VARCHAR(255)	NULLABLE
price	NUMERIC	NULLABLE
location	VARCHAR(255)	NULLABLE
country	VARCHAR(255)	NULLABLE
category	VARCHAR(100)	NULLABLE
lat	NUMERIC	NULLABLE
lng	NUMERIC	NULLABLE
owner_id	INT	FK → USER(user_id)

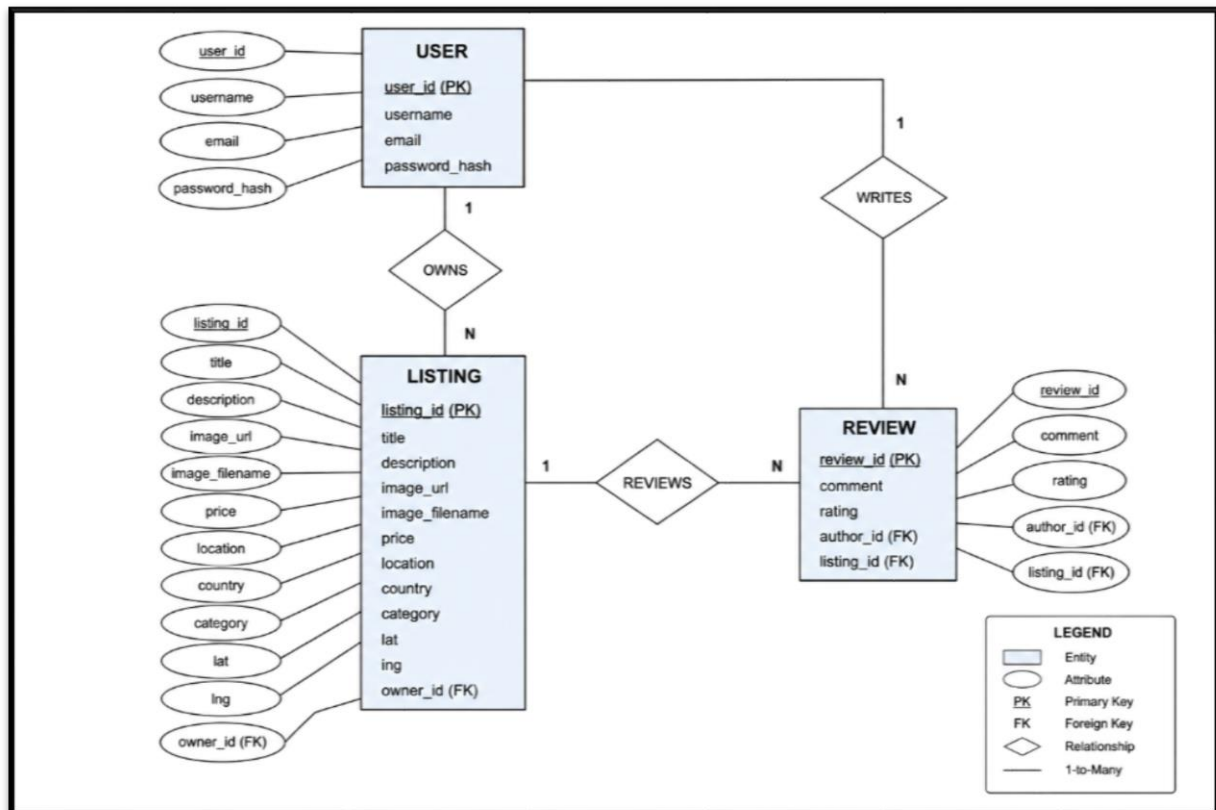
3. REVIEW

Attribute	Data Type	Constraint
review_id	INT / ObjectId	Primary Key
comment	TEXT	NULLABLE
rating	INTEGER (1–5)	CHECK (1 to 5)
author_id	INT	FK → USER(user_id)
listing_id	INT	FK → LISTING(listing_id)

Relationships:

Relationship	Cardinality	Description
USER → LISTING	1 : M	One user (owner) can create many listings
USER → REVIEW	1 : M	One user (author) can write many reviews
LISTING → REVIEW	1 : M	One listing can have many reviews

ER DIAGRAM :



3. ER Analysis

Identifying Entity Sets:

I. USER

- user_id (Primary Key)
- username
- email
- password_hash

II. LISTING

- listing_id (Primary Key)
- title, description, image_url, image_filename
- price, location, country, category
- lat, lng (for Mapbox geolocation)
- owner_id (Foreign Key → USER)

III. REVIEW

- review_id (Primary Key)
- comment, rating
- author_id (Foreign Key → USER)
- listing_id (Foreign Key → LISTING)

Identifying Relationship Sets:

1. Owns — A user creates and owns property listings.

USER → LISTING | Cardinality: USER (1) – LISTING (M)

Participation: Total participation from LISTING side (every listing must have an owner).
Partial from USER (a user may exist without owning any listing).

2. Writes — A user authors reviews for listings.

USER → REVIEW | Cardinality: USER (1) – REVIEW (M)

Participation: Total from REVIEW (every review must have an author). Partial from USER (a user may not have written any review).

3. Has — A listing accumulates multiple reviews from different users.

LISTING → REVIEW | Cardinality: LISTING (1) – REVIEW (M)

Participation: Partial on both sides (a listing may have zero reviews; a user might not review every listing visited).

Special Design Decisions:

- The REVIEW entity carries two foreign keys (author_id and listing_id), making it a weak/associative entity that depends on both USER and LISTING.
- ON DELETE CASCADE is applied from LISTING to REVIEW — deleting a listing automatically removes all its reviews.

- ON DELETE SET NULL is applied from USER to LISTING owner_id — if a user is deleted, listings remain but are disowned.
- ON DELETE CASCADE is applied from USER to REVIEW author_id — deleting a user removes all their reviews.
- The image is stored as two flat columns (image_url, image_filename) rather than a nested object, satisfying 1NF.

4. Conversion of ER to Tables

Each entity in the ER diagram is converted into a relational table. Foreign keys are derived from the relationships between entities.

- USER (user_id, username, email, password_hash)
- LISTING (listing_id, title, description, image_url, image_filename, price, location, country, category, lat, lng, owner_id)

{owner_id} is a foreign key referencing primary key user_id of table USER.

- REVIEW (review_id, comment, rating, author_id, listing_id)

{author_id} is a foreign key referencing primary key user_id of table USER.

{listing_id} is a foreign key referencing primary key listing_id of table LISTING.

Resulting Schema (Visual):

Table	Columns	Foreign Keys
USER	user_id (PK), username, email, password_hash	—
LISTING	listing_id (PK), title, description, image_url, image_filename, price, location, country, category, lat, lng, owner_id	owner_id → USER
REVIEW	review_id (PK), comment, rating, author_id, listing_id	author_id → USER listing_id → LISTING

5. Normalization

Normalization is applied to eliminate redundancy and ensure data consistency. The RentEase database is normalized up to the Third Normal Form (3NF).

1NF – First Normal Form

All attributes in all tables must be atomic (indivisible). There should be no multi-valued or composite attributes.

Example – Non-normalized LISTING table (Before 1NF):

listing_id	title	image (nested)	owner_username	reviews (array)
L001	Villa Solara	{url: '...', filename: 'img1'}	john_doe	[R1, R2, R3] (ObjectIds)

Problems:

- image is a composite/nested object — violates atomicity
- reviews is a multi-valued array attribute — violates 1NF
- owner_username is derived from another table — causes redundancy

Solution – Normalized LISTING table (After 1NF):

listing_id	title	image_url	image_filename	price	owner_id	...
1	Villa Solara	https://...	villa_img.jpg	9000	3	...

Reviews are moved to a separate REVIEW table with listing_id as a foreign key — eliminating the array.

2NF – Second Normal Form

No partial dependencies should exist. Since all tables in RentEase use single-column primary keys (user_id, listing_id, review_id), 2NF is automatically satisfied — every non-key attribute depends on the whole primary key.

Example – Hypothetical non-2NF REVIEW table:

review_id	listing_id	comment	rating	listing_title	author_name
1	5	Great stay!	5	Villa Solara	Ramesh Kumar

Problem: listing_title depends on listing_id (not review_id). author_name depends on author_id (not review_id). Both are partial dependencies.

Solution: Remove listing_title and author_name. Retrieve them via JOIN on REVIEW → LISTING and REVIEW → USER respectively.

3NF – Third Normal Form

No transitive dependencies should exist. Each non-key attribute must depend only on the primary key, not on other non-key attributes.

Example – Hypothetical transitive dependency in LISTING:

listing_id	owner_id	owner_username	owner_email	title	price
1	3	john_doe	john@mail.com	Villa Solara	9000

Problem: listing_id → owner_id → owner_username, owner_email is a transitive dependency. Owner details should not be in LISTING.

Solution: Owner details (username, email) belong only in the USER table. LISTING stores only owner_id as a foreign key.

6. Functional Dependencies after Normalization

Table	Functional Dependency
USER	user_id → username, email, password_hash
LISTING	listing_id → title, description, image_url, image_filename, price, location, country, category, lat, lng, owner_id
REVIEW	review_id → comment, rating, author_id, listing_id

All tables are in 3NF. No partial or transitive dependencies exist. Each non-key attribute is fully functionally dependent on its respective primary key only.

7. SQL Queries (DDL & DML)

Creating Tables (DDL):

Create USER table:

```
CREATE TABLE IF NOT EXISTS users (  
    user_id    SERIAL PRIMARY KEY,  
    username   VARCHAR(255) NOT NULL UNIQUE,  
    email      VARCHAR(255) NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL  
);
```

Create LISTING table:

```
CREATE TABLE IF NOT EXISTS listings (  
    listing_id    SERIAL PRIMARY KEY,  
    title         VARCHAR(255) NOT NULL,  
    description    TEXT,  
    image_url     TEXT DEFAULT 'https://images.unsplash.com/...',  
    image_filename VARCHAR(255),  
    price         NUMERIC,  
    location      VARCHAR(255),  
    country       VARCHAR(255),  
    category      VARCHAR(100),  
    lat           NUMERIC,  
    lng           NUMERIC,  
    owner_id      INTEGER NULL REFERENCES users(user_id) ON DELETE SET NULL  
);
```

Create REVIEW table:

```
CREATE TABLE IF NOT EXISTS reviews (  
    review_id    SERIAL PRIMARY KEY,  
    comment      TEXT,  
    rating       INTEGER CHECK (rating BETWEEN 1 AND 5),  
    author_id    INTEGER REFERENCES users(user_id) ON DELETE CASCADE,  
    listing_id   INTEGER REFERENCES listings(listing_id) ON DELETE CASCADE  
);
```

Inserting Sample Data (DML):

Insert into USERS:

```
INSERT INTO users (username, email, password_hash)  
VALUES ('abhinav_p', 'abhinav@tiet.ac.in', 'hashed_pass_1');  
  
INSERT INTO users (username, email, password_hash)  
VALUES ('priya_s', 'priya@example.com', 'hashed_pass_2');  
  
INSERT INTO users (username, email, password_hash)  
VALUES ('rahul_k', 'rahul@example.com', 'hashed_pass_3');
```

Insert into LISTINGS:

```
INSERT INTO listings (title, description, price, location, country, category, lat, lng, owner_id)
VALUES ('Villa Solara', 'Luxury villa with pool', 9000, 'Jaipur', 'India', 'Resort', 26.91, 75.78, 1);
```

```
INSERT INTO listings (title, description, price, location, country, category, lat, lng, owner_id)
VALUES ('Cozy Beachfront Cottage', 'Sea-facing cottage', 12000, 'Goa', 'India', 'Apartment', 15.29, 73.91, 2);
```

```
INSERT INTO listings (title, description, price, location, country, category, lat, lng, owner_id)
VALUES ('Leh Ladakh', 'Mountain retreat', 1500, 'Leh', 'India', 'Houseboat', 34.15, 77.57, 1);
```

```
INSERT INTO listings (title, description, price, location, country, category, lat, lng, owner_id)
VALUES ('Rustic Cabin by the Lake', 'Peaceful lakeside cabin', 13981, 'Nainital', 'India', 'Apart Hotel', 29.38, 79.46, 3);
```

Insert into REVIEWS:

```
INSERT INTO reviews (comment, rating, author_id, listing_id)
VALUES ('Absolutely beautiful property!', 5, 2, 1);
```

```
INSERT INTO reviews (comment, rating, author_id, listing_id)
VALUES ('Great location, very clean.', 4, 3, 1);
```

```
INSERT INTO reviews (comment, rating, author_id, listing_id)
VALUES ('Amazing mountain views!', 5, 1, 3);
```

```
INSERT INTO reviews (comment, rating, author_id, listing_id)
VALUES ('Value for money, loved it.', 4, 2, 4);
```

Analytical SQL Queries:**1. All listings with their owner username:**

```
SELECT l.title, l.price, l.location, l.category, u.username AS owner
FROM listings l
JOIN users u ON l.owner_id = u.user_id
ORDER BY l.price DESC;
```

2. Average rating per listing:

```
SELECT l.title, l.location,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(r.review_id) AS total_reviews
FROM listings l
LEFT JOIN reviews r ON r.listing_id = l.listing_id
GROUP BY l.listing_id, l.title, l.location
```

```
ORDER BY avg_rating DESC;
```

3. Top-rated listings (rating >= 4.5):

```
SELECT l.title, l.category, l.price,  
       ROUND(AVG(r.rating), 2) AS avg_rating  
FROM listings l  
JOIN reviews r ON r.listing_id = l.listing_id  
GROUP BY l.listing_id, l.title, l.category, l.price  
HAVING AVG(r.rating) >= 4.5  
ORDER BY avg_rating DESC;
```

4. Most active reviewers:

```
SELECT u.username, COUNT(r.review_id) AS reviews_written  
FROM users u  
JOIN reviews r ON r.author_id = u.user_id  
GROUP BY u.user_id, u.username  
ORDER BY reviews_written DESC;
```

5. Listings with no reviews:

```
SELECT l.title, l.location, l.price  
FROM listings l  
LEFT JOIN reviews r ON r.listing_id = l.listing_id  
WHERE r.review_id IS NULL;
```

6. Listings per category with average price:

```
SELECT category,  
       COUNT(*) AS total_listings,  
       ROUND(AVG(price), 2) AS avg_price  
FROM listings  
GROUP BY category  
ORDER BY total_listings DESC;
```

7. Most expensive listing per country:

```
SELECT country, title, price  
FROM listings l1  
WHERE price = (  
    SELECT MAX(price) FROM listings l2  
    WHERE l2.country = l1.country  
)  
ORDER BY price DESC;
```

8. Table Structure

USERS:

user_id	username	email	password_hash
1	abhinav_p	abhinav@tiet.ac.in	\$2b\$10\$abc...xyz
2	vaibhav_s	vaibhav@tiet.ac.in	\$2b\$10\$def...uvw
3	rahul_k	rahul@example.com	\$2b\$10\$ghi...rst

LISTINGS:

id	title	price	location	country	category	owner_id
1	Villa Solara	9,000	Jaipur	India	Resort	1
2	Cozy Beachfront Cottage	12,000	Goa	India	Apartment	2
3	Leh Ladakh	1,500	Leh	India	Houseboat	1
4	Rustic Cabin by Lake	13,981	Nainital	India	Apart Hotel	3
5	Secluded Treehouse	21,000	Coorg	India	Resort	2
6	Silver Leaf Retreat	45,000	Udaipur	India	Apart Hotel	3

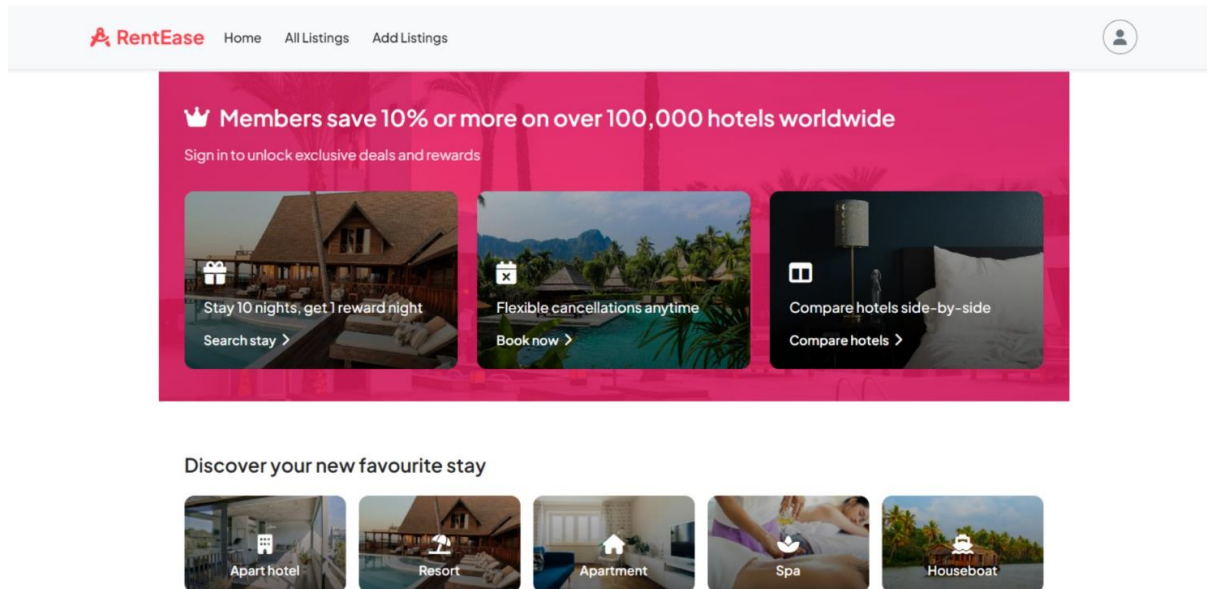
REVIEWS:

id	comment	rating	author_id	listing_id
1	Absolutely beautiful property!	5	2	1
2	Great location, very clean.	4	3	1
3	Amazing mountain views!	5	1	3
4	Value for money, loved it.	4	2	4
5	Perfect getaway, highly recommend!	5	3	5

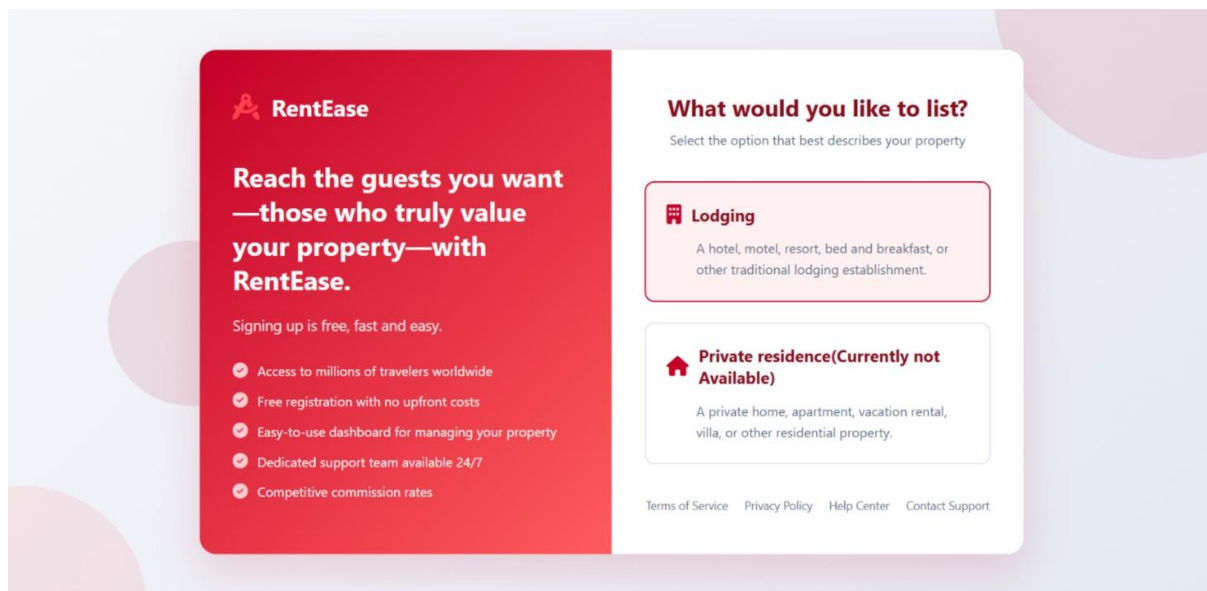
9. Project Screenshots

The following screenshots demonstrate the live functionality of the RentEase web application deployed on Render. The platform includes all listings, categories, features section, FAQ, and footer.

Screenshot 1: All Listings Page — Hotel Cards with Pricing



Screenshot 2: Features Section & FAQ — Book with Confidence, Amenities, Flexible Stays



Screenshot 3: FAQ Section & Footer — Questions Answered, Footer Links

Your questions, answered

▼ What is RentEase and how does it work?



RentEase connects people who want to rent out their homes with people looking for accommodations.

▼ How do I use search filters?



Use the filter options to narrow down listings by price, amenities, location, and more.

▼ Do I need to meet my Host?



Some hosts offer self check-in, others prefer a personal welcome. It depends on the listing.

▼ What if I need to cancel due to a problem with the listing or Host?



We offer flexible cancellation options and support in case something goes wrong.

▼ Need more information?



You can always reach out to our Help Center or contact us directly.

10. PL/SQL Components

This section covers PL/SQL constructs implemented for the RentEase database — including stored procedures, functions, triggers, cursors, and exception handling. These components automate business logic, enforce data rules, and enable efficient data processing directly at the database layer.

10.1 Stored Procedure

Procedure: Add a new listing with owner validation

This procedure inserts a new listing into the listings table only if the given owner (`user_id`) exists in the users table. If the user does not exist, it raises a custom exception and rolls back the operation.

```
CREATE OR REPLACE PROCEDURE add_listing(
  p_title          VARCHAR,
  p_description     TEXT,
  p_price          NUMERIC,
  p_location       VARCHAR,
  p_country        VARCHAR,
  p_category       VARCHAR,
  p_owner_id      INTEGER
) LANGUAGE plpgsql AS $$
DECLARE
  v_count INTEGER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM users WHERE user_id = p_owner_id;
  IF v_count = 0 THEN
    RAISE EXCEPTION 'Owner with user_id % does not exist.', p_owner_id;
  END IF;
  INSERT INTO listings (title, description, price, location, country, category,
owner_id)
  VALUES (p_title, p_description, p_price, p_location, p_country, p_category,
p_owner_id);
  RAISE NOTICE 'Listing "%" added successfully.', p_title;
EXCEPTION
  WHEN OTHERS THEN
    RAISE NOTICE 'Error: %', SQLERRM;
END;
$$;

-- Call the procedure
CALL add_listing('Sunset Villa', 'Sea-view luxury villa', 15000, 'Mumbai', 'India',
'Resort', 1);
```

10.2 Function

Function: Get average rating of a listing

This function accepts a `listing_id` and returns its average review rating rounded to two decimal places. It returns NULL if the listing has no reviews.

```
CREATE OR REPLACE FUNCTION get_avg_rating(p_listing_id INTEGER)
RETURNS NUMERIC LANGUAGE plpgsql AS $$
DECLARE
  v_avg NUMERIC;
```

```

BEGIN
    SELECT ROUND(AVG(rating), 2) INTO v_avg
    FROM reviews WHERE listing_id = p_listing_id;
    RETURN v_avg;
END;
$$;

-- Call the function
SELECT get_avg_rating(1); -- Returns avg rating for listing_id = 1

```

10.3 Trigger

Trigger: Prevent duplicate reviews by the same user on the same listing

This BEFORE INSERT trigger fires on the reviews table and checks whether the same user (author_id) has already reviewed the same listing (listing_id). If a duplicate is detected, the insert is aborted with a descriptive error message.

```

CREATE OR REPLACE FUNCTION check_duplicate_review()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_exists INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_exists FROM reviews
    WHERE author_id = NEW.author_id AND listing_id = NEW.listing_id;
    IF v_exists > 0 THEN
        RAISE EXCEPTION 'User % has already reviewed listing %.', NEW.author_id,
NEW.listing_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_no_duplicate_review
BEFORE INSERT ON reviews
FOR EACH ROW EXECUTE FUNCTION check_duplicate_review();

```

10.4 Cursor

Cursor: Iterate over all listings and print their average rating

This anonymous PL/pgSQL block uses an explicit cursor to loop through every listing and uses the get_avg_rating() function to print its average rating. It demonstrates row-by-row processing using FETCH and CLOSE.

```

DO $$
DECLARE
    CURSOR c_listings IS SELECT listing_id, title FROM listings;
    v_id    INTEGER;
    v_title VARCHAR;
    v_avg   NUMERIC;
BEGIN
    OPEN c_listings;
    LOOP
        FETCH c_listings INTO v_id, v_title;
        EXIT WHEN NOT FOUND;
        v_avg := get_avg_rating(v_id);
        RAISE NOTICE 'Listing: % | Avg Rating: %', v_title, COALESCE(v_avg::TEXT, 'No
reviews');
    END LOOP;
END;

```

```
END LOOP;
CLOSE c_listings;
END;
$$;
```

10.5 Exception Handling

This block demonstrates structured exception handling using SQLSTATE codes. It attempts to insert a review with an invalid rating (6, which violates the CHECK constraint) and catches the resulting error gracefully without crashing the session.

```
DO $$
BEGIN
    INSERT INTO reviews (comment, rating, author_id, listing_id)
    VALUES ('Invalid rating test', 6, 1, 1); -- rating 6 violates CHECK (1-5)
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'CHECK violation: Rating must be between 1 and 5. Got: 6';
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'Foreign key violation: author or listing does not exist.';
    WHEN OTHERS THEN
        RAISE NOTICE 'Unexpected error: % (SQLSTATE: %)', SQLERRM, SQLSTATE;
END;
$$;
```

11. Conclusion

The RentEase Hotel Listing & Management System has been successfully designed and implemented using a relational database model. The schema includes well-normalized tables for users, listings, and reviews. Referential integrity is maintained through primary and foreign key constraints, ensuring data consistency across the system.

The database design supports the core functionalities of the RentEase platform including user authentication, property listing management, and review tracking. The ER model clearly captures the three main entities and their relationships with proper cardinality constraints.

By normalizing the database to 3NF, the design eliminates redundancy and ensures that every non-key attribute depends directly on the primary key. Key design decisions include the separation of the image object into flat columns (`image_url`, `image_filename`) to satisfy 1NF, and the removal of the reviews array from the listing document in favor of a dedicated REVIEW table with foreign key references.

The analytical SQL queries demonstrate the system's capability to generate meaningful insights such as top-rated listings, most active reviewers, category-wise pricing, and listings without reviews — all of which are valuable for both platform administrators and end users.

This project stands as a practical demonstration of core DBMS concepts — ER modeling, normalization, DDL/DML operations, and analytical querying — applied to a real-world full-stack web application.

12. References

1. Ramez Elmasri, Shamkant B. Navathe. Fundamentals of Database Systems, 7th Edition. Pearson Education, 2016.
2. Abraham Silberschatz, Henry F. Korth, S. Sudarshan. Database System Concepts, 7th Edition. McGraw-Hill, 2019.
3. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
4. GeeksforGeeks – DBMS Tutorials. <https://www.geeksforgeeks.org/dbms/>
5. W3Schools SQL Tutorial. <https://www.w3schools.com/sql/>
6. Node.js Documentation. <https://nodejs.org/en/docs/>
7. Express.js Documentation. <https://expressjs.com/>
8. GitHub Repository – RentEase. <https://github.com/VaibhavSinghal0159/rentease>